

Table of Contents

Introduction	2
Partie I — Rapport	6
A. Fondements théoriques — La méthode SVN	7
B. Planification et déroulement du projet	10
C. Conception et décisions d'architecture	13
D. Tests et validation	16
Partie II — Spécification technique	22
1. Description générale	23
2. Besoins fonctionnels	31
3. Spécification des structures de données	41
4. Spécifications des interfaces externes	49
5. Avancement du projet	53
Références	58
Index	60
Annexes	66

Introduction

Ce projet s'inscrit dans le contexte de l'UV DE50 du semestre P26.

Il a été réalisé par **Guillaume RETTER**, **Hugo CHARCOT**, **Arnaud MICHEL**, **Benjamin GROISNE** et **Antoine LAURANT** sous la supervision de **M. V. HILAIRE**

Objectifs et apport du projet

IBM Rational Rhapsody est un environnement de référence pour la modélisation de systèmes complexes. Il permet aux ingénieurs systèmes de concevoir leurs architectures en SysML/UML dans un cadre MBSE (Model-Based Systems Engineering). Cependant, Rhapsody ne propose aucun support natif de la méthode **Stakeholder Value Network (SVN)**.

Le projet **RhapsodySVN** vise à résoudre ce problème en développant un **plugin Java pour IBM Rhapsody 9.0** qui intègre nativement la méthode SVN dans l'environnement de modélisation. Concrètement, le plugin permet de :

- **modéliser** graphiquement les arcs de valeur entre acteurs et système directement dans un diagramme Rhapsody, sans recourir à un outil externe ;
- **quantifier** automatiquement l'importance relative de chaque partie prenante selon les équations de Cameron (2007), avec mise à jour en temps réel à chaque modification du modèle ;
- **informer** les choix d'architecture en propageant les scores calculés directement dans les éléments du modèle SysML, sous forme de tag values.

Structure du document

Ce rapport est organisé en deux parties distinctes :

- **Partie I — Rapport:** présente les fondements théoriques de la méthode SVN, la planification et le déroulement du projet, les décisions d'architecture, et la stratégie de tests.
- **Partie II — Spécification technique:** décrit le système de façon formelle — description générale, besoins fonctionnels, structures de données, interfaces et avancement du

projet.

Définitions et abréviations

Terme / Abréviation	Définition
SVN	Stakeholder Value Network — méthode d'analyse des parties prenantes et de leurs échanges de valeur
SysML	Systems Modeling Language — langage de modélisation de systèmes basé sur UML
UML	Unified Modeling Language — langage de modélisation objet standard
IBM Rhapsody	Outil de modélisation SysML/UML développé par IBM (Rational)
MBSE	Model-Based Systems Engineering — ingénierie système pilotée par le modèle
INCOSE	International Council on Systems Engineering
API	Application Programming Interface
JDK	Java Development Kit
JAR	Java ARchive — format d'emballage d'applications Java
UC	Use Case — cas d'utilisation
DFS	Depth-First Search — algorithme de parcours en profondeur d'un graphe
Value Loop	Cycle dans le graphe SVN passant par le nœud système central

RPUserPlugin	Interface Java de l'API Rhapsody permettant de déclarer un plugin
IRPApplication	Interface Rhapsody représentant l'application en cours d'exécution
IRPProject	Interface Rhapsody représentant le projet ouvert
IRPActor	Interface Rhapsody représentant un acteur
IRPDependency	Interface Rhapsody représentant une dépendance UML (utilisée pour les arcs «valuearc»)
Stereotype	Extension du métamodèle UML/SysML permettant de typer un élément
Tag	Propriété supplémentaire associée à un élément via un stéréotype

Partie I — Rapport

Cette première partie constitue le rapport de projet au sens traditionnel du terme. Elle retrace le contexte l'évolution du projet ainsi que les choix organisationnels et techniques qui ont guidé son déroulement.

A. Fondements théoriques — La méthode SVN

Qu'est-ce qu'un Stakeholder Value Network ?

Un **Stakeholder Value Network (SVN)** est une méthode d'analyse des parties prenantes proposée par Cameron (2007) ("[Références bibliographiques](#)" in "[Références](#)"). Elle modélise les relations entre un système et ses parties prenantes sous la forme d'un **graphe orienté pondéré**, dans lequel chaque arc représente un flux de valeur entre deux entités.

Contrairement aux approches classiques qui se contentent d'identifier les parties prenantes et leurs besoins, la méthode SVN cherche à **quantifier l'importance relative** de chaque partie prenante en tenant compte de la structure des échanges de valeur au sein du réseau. Cette quantification permet d'établir un classement objectif des parties prenantes, utile pour prioriser les exigences et guider les arbitrages architecturaux.

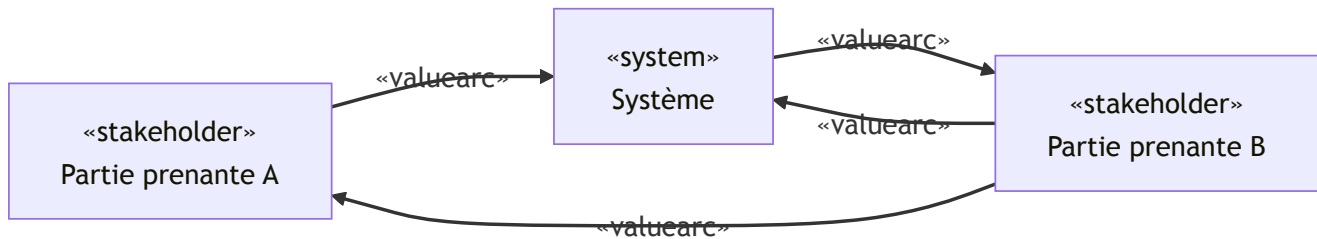
Dans une démarche MBSE, le SVN se positionne en **amont de la définition des exigences**: il fournit une base quantitative pour déterminer quelles parties prenantes doivent être satisfaites en priorité, et donc quelles fonctions du système sont les plus critiques.

Structure d'un graphe SVN

Un réseau SVN est composé de trois types d'entités :

- **Le nœud «system»**: représente le système en cours de conception. C'est le nœud central autour duquel gravitent les parties prenantes. Il joue un rôle particulier dans le calcul des scores.
- **Les nœuds «stakeholder»**: représentent les parties prenantes (acteurs humains ou organisations) qui interagissent avec le système. Chaque stakeholder reçoit un score d'importance calculé par la méthode.
- **Les arcs «valuearc»**: représentent les flux de valeur orientés entre entités. Chaque arc est pondéré par deux critères : le **benefitRanking** (importance du bénéfice pour le destinataire) et le **supplyImportance** (capacité du fournisseur à satisfaire ce flux).

Exemple d'un graphe SVN



Les value loops

Le concept central de la méthode est la **value loop**: un cycle dans le graphe SVN qui passe par le nœud «system». Une value loop représente un circuit de valeur complet — le système reçoit quelque chose d'un acteur et le restitue (directement ou indirectement) à un ou plusieurs autres acteurs.

Plus un stakeholder participe à de nombreuses value loops, et plus ces loops ont un score élevé, plus ce stakeholder est considéré comme important pour le système. La méthode de Cameron formalise cette intuition à travers deux équations.

Calcul de l'importance — Équations de Cameron (2007)

Score d'un arc

Chaque arc «valuearc» reçoit un score numérique déterminé par la combinaison de ses deux attributs de pondération. La matrice ci-dessous, issue de la Figure 3 de Sease et al. (INCOSE 2018) (["Références bibliographiques" in "Références"](#)), définit ces scores :

supplyImportance ↓ \ benefitRanking →	MIGHT_BE	SHOULD_BE	MUST_BE
HIGH	0.30	0.50	0.95
MEDIUM	0.20	0.40	0.80
LOW	0.10	0.20	0.40

Les valeurs par défaut à la création d'un arc sont **MIGHT_BE** et **LOW**, soit un score de 0.10.

Équation 1 — Score d'une value loop

Le score d'une value loop est le **produit des scores de tous ses arcs**:

$$\text{score}(L) = \prod_i \text{score}(\text{arc}_i)$$

Un seul arc de faible valeur dans une loop suffit à réduire significativement son score global, ce qui reflète l'idée qu'une chaîne de valeur n'est aussi forte que son maillon le plus faible.

Équation 2 — Importance d'un stakeholder

L'importance d'un stakeholder S est calculée comme la **fraction des scores de loops qui le contiennent**, rapportée au score total de toutes les loops du réseau (Cameron 2007 (["Références bibliographiques" in "Références"](#)), Équation 2) :

$$\text{importance}(S) = \frac{\sum_{L \ni S} \text{score}(L)}{\sum_L \text{score}(L)}$$

Le résultat est un score normalisé entre 0 et 1. Un stakeholder présent dans toutes les loops de fort score obtiendra un score proche de 1 ; un stakeholder isolé ou présent uniquement dans des loops de faible valeur obtiendra un score proche de 0.

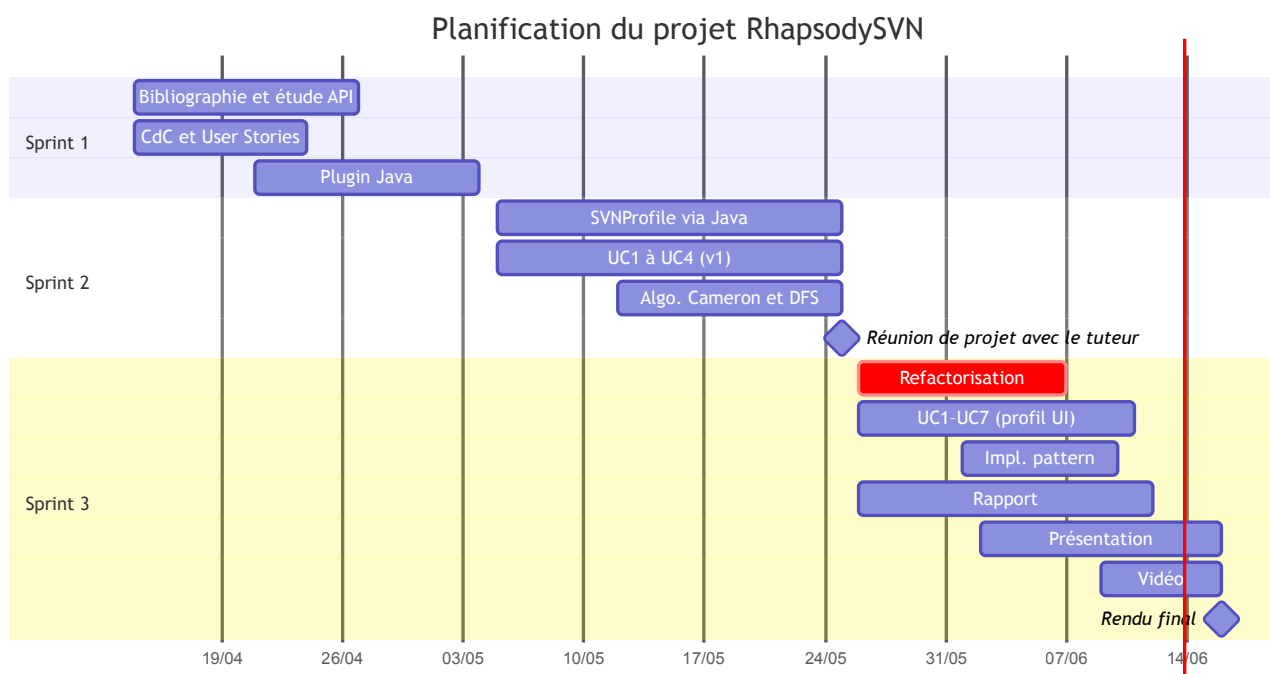
B. Planification et déroulement du projet

Organisation générale

Le projet a été conduit selon une méthodologie **Agile Scrum**, découpée en **3 sprints de 3 semaines** du 14 avril au 16 juin 2026. Chaque sprint est encadré par un sprint planning en ouverture et une rétrospective en clôture. Le backlog est géré sur **ClickUp**.

Rôle	Membre
Product Owner	Guillaume Retter
Scrum Master	Arnaud Michel
Développeurs	Hugo Charcot, Benjamin Groisne, Antoine Laurant

Diagramme de Gantt



Déroulement par sprint

Sprint 1 — Exploration et mise en place (14 avril – 5 mai)

Le premier sprint a été consacré à l'installation et la prise en main de Rhapsody.

L'équipe a également débuté par réaliser une phase de recherche sur la méthode SVN et d'étude approfondie de l'API Java COM d'IBM Rhapsody, peu documentée. En parallèle, le cahier des charges et les premières User Stories ont été rédigés.

Un **POC** a été réalisé en fin de sprint : tentative de création du plugin et du profil SVN entièrement par code Java. Cette exploration a révélé une limitation importante de l'API — il est impossible de créer de nouveaux types de diagrammes depuis Java seul. Ce constat a orienté les choix architecturaux du sprint suivant.

Sprint 2 — Première implémentation (5 mai – 26 mai)

Le deuxième sprint a produit une première version fonctionnelle du plugin. Le profil `SVNProfile` (stéréotypes, types énumérés, tag values) a été créé via l'API Java, et les quatre premiers cas d'utilisation (UC1 à UC4) ([2. Besoins fonctionnels](#)) ont été implémentés en version initiale, incluant l'algorithme DFS de détection des value loops et le calcul des scores de Cameron.

En fin de sprint, après un entretien avec l'enseignant référent, un retour important a conduit à un changement majeur architectural : il a été recommandé de définir le profil SVN depuis l'interface graphique de Rhapsody plutôt que par code. Cette approche offre une meilleure compatibilité et facilite la maintenance. Ce changement a été intégré comme tâche critique du Sprint 3.

Sprint 3 — Refactorisation, architecture réactive et livrables (26 mai – 16 juin)

Le troisième sprint a été le plus dense. Il a débuté par une **refactorisation majeure**:

- suppression de la création programmatique du profil
- réimplémentation de l'ensemble des UC sur la base du profil défini dans l'interface Rhapsody.

L'architecture a simultanément évolué vers un modèle **event-driven** avec l'introduction du `Listener` (`RPAApplicationListener`), qui déclenche automatiquement le recalcul à

chaque modification du modèle. Le **pattern Strategy** a été mis en place pour rendre les algorithmes de calcul interchangeables. Des **tests unitaires** ont été ajoutés pour couvrir la logique algorithmique.

Le sprint s'est conclu par la production des livrables finaux : le présent rapport, les supports de présentation ainsi que la vidéo explicative.

C. Conception et décisions d'architecture

Vue d'ensemble

Le plugin RhapsodySVN est structuré en couches avec une séparation claire des responsabilités. Le diagramme de classes ci-dessous illustre les relations entre les composants principaux :

 [Insérer ici le diagramme de classes]

Décisions architecturales

Architecture réactive plutôt que commandes manuelles

La version initiale du plugin reposait sur un ensemble de **commandes manuelles** déclenchées depuis le menu Rhapsody : l'utilisateur devait explicitement lancer le calcul, mettre à jour les labels, etc. Cette approche présentait un risque important d'incohérence : après toute modification du modèle, le plugin restait dans un état obsolète jusqu'à la prochaine exécution manuelle.

À partir du Sprint 3, l'architecture a basculé vers un modèle **event-driven** s'appuyant sur la classe `Listener` qui étend `RPAApplicationListener`. Ce listener est enregistré auprès de Rhapsody au démarrage du plugin et reçoit automatiquement les notifications pour deux événements :

- `afterAddElement` — déclenché lorsqu'un nouvel élément est ajouté au modèle
- `onElementsChanged` — déclenché lorsque la valeur d'un tag est modifiée ou qu'un élément est supprimé

Grâce à cette architecture, **toute modification du modèle provoque immédiatement un recalcul**, sans intervention de l'utilisateur. Cela élimine les risques d'incohérence et simplifie considérablement l'expérience d'utilisation.

Pattern Strategy pour les algorithmes de calcul

Le calcul de l'importance des stakeholders peut être conduit selon deux algorithmes distincts :

- **ValueLoopStrategy** — implémente les équations de Cameron (2007) (["Références bibliographiques" in "Références"](#)) via une recherche DFS des value loops. Nécessite la présence d'un nœud «system» dans le modèle.
- **ArcSumStrategy** — stratégie de secours qui calcule l'importance comme la somme normalisée des scores des arcs connectés à chaque stakeholder. Utilisée en l'absence de nœud «system».

Ces deux algorithmes sont définis derrière l'interface **ICalculationStrategy**. Le **CalculationService** choisit la stratégie appropriée à l'exécution, sans que le **Listener** ni le reste du code ne soient affectés par ce choix. Ce pattern offre deux avantages concrets : il facilite l'ajout de nouvelles stratégies à l'avenir, et il permet de tester chaque algorithme indépendamment.

Modélisation des arcs via **IRPDependency**

Les arcs «valuearc» sont modélisés comme des **IRPDependency** (dépendances UML génériques) plutôt que comme des **IRPConnector** ou des associations de blocs IBD/BDD. Ce choix est contraint par les **limitations de l'API Java de Rhapsody**. En effet, la création de connecteurs SysML structurels depuis du code Java n'est pas possible sans passer par un profil IBD dédié, ce qui aurait complexifié considérablement l'implémentation.

L'utilisation d' **IRPDependency** permet de créer des arcs orientés entre n'importe quels éléments du modèle (acteurs et classes), d'y attacher des stéréotypes et des tags, et de les faire apparaître dans un diagramme **ObjectModelDiagram**. En contrepartie, ces arcs s'éloignent de la sémantique SysML stricte attendue pour un diagramme IBD.

Isolation de l'API Rhapsody

L'ensemble des appels à l'API IBM Rhapsody est concentré dans deux classes du package **rhapsody**:

- **RhapsodyWrapper** — accès bas niveau en lecture et en écriture (tags, stéréotypes, collections d'éléments)

- **RhapsodyElementUpdater** — mise à jour des éléments du modèle (labels d'arcs, scores de stakeholders)

Cette isolation limite la surface de code à modifier en cas de changement de version de Rhapsody. Le reste de l'architecture (stratégies, service de calcul, modèles) est totalement indépendant de l'API et peut être testé sans instance Rhapsody.

Design patterns mis en œuvre

Patte rn	Classe principal e	Justification
Obse rver	Listener (RPAp plicationListener)	Souscrit aux événements Rhapsody pour déclencher auto matiquement le recalcul sans polling ni intervention utiliza teur
Strat egy	ICalculationStra tegy	Sépare le choix de l'algorithme de son utilisation ; permet d'ajouter une stratégie sans modifier CalculationService
Singl eton	Logger	Garantit une instance unique accessible depuis tous les c omposants sans passer de référence explicite

D. Tests et validation

 **Note:** partie à modifier

Stratégie de test

La stratégie de test adoptée sur ce projet est **centrée sur la logique métier pure**, c'est-à-dire les composants indépendants de l'API IBM Rhapsody. Les couches d'accès à Rhapsody (`Listener`, `RhapsodyWrapper`, `RhapsodyElementUpdater`) ne peuvent pas être testées sans une instance en cours d'exécution de l'application IBM, qui nécessite une licence et un environnement Windows dédié. Ces couches sont donc exclues des tests automatisés.

La couverture automatisée porte sur trois domaines :

- **Le modèle de données** — score des arcs (`ValueArc`), score des loops (`ValueLoop`), état du parcours DFS (`SearchState`)
- **L'algorithme de détection des loops** — parcours DFS, construction des cycles
- **Le calcul d'importance** — implémentation des équations de Cameron via `ValueLoopStrategy`

Les tests sont écrits avec **JUnit 4** et placés dans `src/test/java/fr/utbm/svn/`.

Tests du modèle de données

Matrice de score des arcs — `ValueArcScoreTest`

Ce fichier de test couvre l'ensemble des 9 cellules de la matrice de score définie à la section A. Fondements théoriques ([A. Fondements théoriques — La méthode SVN](#)), plus le cas du score par défaut pour des valeurs inconnues.

Combinaison testée	Score attendu
HIGH/MUST_BE	0.95
MEDIUM/MUST_BE	0.80
LOW/MUST_BE	0.40
HIGH/SHOULD_BE	0.50
MEDIUM/SHOULD_BE	0.40
LOW/SHOULD_BE	0.20
HIGH/MIGHT_BE	0.30
MEDIUM/MIGHT_BE	0.20
LOW/MIGHT_BE	0.10
Valeurs inconnues (défaut)	0.20

Ces tests garantissent la conformité de l'implémentation avec les valeurs de la matrice et documentent le comportement du chemin par défaut (score 0.20 renvoyé lorsque les attributs ne correspondent à aucune combinaison connue).

Modèle ValueLoop — ValueLoopTest

Ces tests vérifient le comportement du conteneur de value loop, notamment le calcul du score via l' **Équation 1** (produit des scores des arcs) :

Test	Ce qui est vérifié
<code>constructor_storeArcs</code>	Le constructeur stocke correctement la liste des arcs
<code>getScore_returnsZero_initially</code>	Le score vaut 0 en l'absence de calcul préalable
<code>getScore_singleArc</code>	Loop à un seul arc : score = score de l'arc
<code>getScore_multipleArcs_product</code>	Loop à plusieurs arcs : score = produit des scores individuels
<code>getScore_afterUpdate</code>	Mise à jour du score après modification des arcs

Le test multi-arcs est particulièrement important : il valide que l'implémentation applique bien le **produit** et non une somme. Par exemple, pour une loop de scores `[0.5, 0.4, 0.8]`, le score attendu est $0.5 \times 0.4 \times 0.8 = 0.16$.

État du parcours DFS — SearchStateTest

`SearchState` est le conteneur immuable qui représente un état intermédiaire de la recherche DFS lors de la détection des value loops. Il stocke :

- le **nœud courant** du parcours,
- le **chemin parcouru** depuis le nœud de départ,
- les **scores accumulés** sur le chemin,
- l'ensemble des **nœuds visités**.

Test	Ce qui est vérifié
<code>constructor_storesAllFields</code>	Tous les champs sont correctement initialisés
<code>currentField</code>	L'accesseur <code>getCurrent()</code> renvoie le nœud courant
<code>emptyFields</code>	Comportement correct avec des listes et ensembles vides

Ces tests garantissent l'intégrité de l'état transmis d'une itération DFS à la suivante.

Tests d'intégration algorithmique — ImportanceCalculationTest

Ce fichier de test valide l'ensemble de la chaîne de calcul : détection des loops par DFS + calcul de l'importance via l' **Équation 2** de Cameron. Il utilise des graphes construits en mémoire (sans Rhapsody) et appelle directement la stratégie `ValueLoopStrategy`.

Test	Graphe	Résultat attendu
singleLoop	$S \rightarrow SH1 \rightarrow S$ (1 loop, 1 arc de score connu)	Score stakeholder = 1.0 (seule loop du réseau)
twoLoops_stakeholderInBoth	$S \rightarrow SH1 \rightarrow S$ et $S \rightarrow SH2 \rightarrow S$ $H1 \rightarrow S$	SH1 présent dans les 2 loops, SH2 dans 1 seule
noLoop_returnsZero	Graphe acyclique sans nœud «system»	Score = 0 pour tous les stakeholders
threeNodeLoop	$S \rightarrow SH1 \rightarrow SH2 \rightarrow S$ (loop de 3 nœuds)	Score = produit des 3 arcs
stakeholderNotInLoop	SH2 non connecté au cycle principal	Importance(SH2) = 0
emptyGraph	Aucun nœud	Aucune exception, résultat vide

Le test `noLoop_returnsZero` couvre un cas d'usage réel : si l'utilisateur n'a pas encore ajouté de nœud «system» dans son modèle, aucune loop ne peut être détectée et tous les scores restent à zéro. Ce comportement est géré proprement sans erreur.

Le test `emptyGraph` valide la robustesse de l'algorithme face à un modèle vide, cas pouvant survenir au démarrage du plugin avant tout ajout d'éléments.

Limites de la couverture

Composant	Couverture	Raison
ValueArc.getArcScore()	Complète (10/10 cas)	Logique pure, sans dépendance externe
ValueLoop.getScore()	Complète	Logique pure
SearchState	Complète	Structure de données simple
ValueLoopStrategy	Fonctionnelle (6 scénarios)	Testable avec graphes synthétiques
ArcSumStrategy	Non couverte	Stratégie de secours, logique simple non critique
Listener	Non couverte	Nécessite une instance Rhapsody en cours d'exécution
RhapsodyWrapper	Non couverte	API Rhapsody non mocable sans licence
RhapsodyElementUpdater	Non couverte	API Rhapsody non mocable sans licence
CalculationService	Non couverte	Orchestrateur dépendant de Rhapsody

L'isolation de l'API Rhapsody dans les classes dédiées (voir section C. Conception et architecture ([C. Conception et décisions d'architecture](#))) a été la condition préalable à la mise en place de ces tests : en concentrant tous les appels à Rhapsody dans RhapsodyWrapper et RhapsodyElementUpdater, les algorithmes métier (ValueLoopStrategy, ValueLoop, ValueArc) peuvent être testés indépendamment de toute infrastructure IBM.

Partie II — Spécification technique

Cette partie constitue la spécification technique du projet. Elle décrit formellement ce que le plugin RhapsodySVN doit faire, comment il est structuré, et à quelles améliorations futures peuvent être ajoutées.

1. Description générale

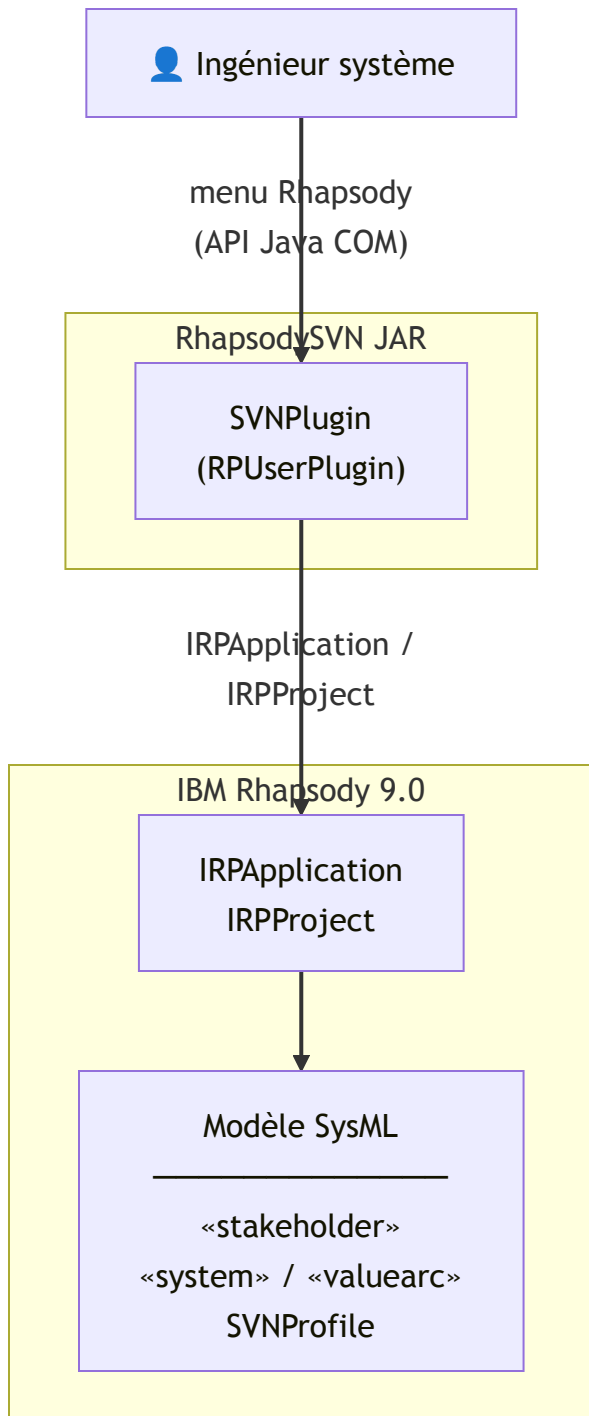
Environnement ou contexte du système

Le plugin **RhapsodySVN** est un composant logiciel qui s'exécute au sein de l'environnement **IBM Rhapsody 9.0**. Il s'intègre à Rhapsody via la classe **SVNPlugin**, qui implémente l'interface **RPUserPlugin** de l'API Java COM de Rhapsody.

Il interagit avec trois entités principales :

- **IBM Rhapsody**: l'application hôte, avec laquelle le plugin communique via l'API Java COM (**RhapsodyAppServer**). Rhapsody doit être ouvert et un projet actif doit être chargé pour que le plugin fonctionne. Le plugin s'intègre au menu contextuel de Rhapsody via la classe **SVNPlugin**.
- **Le modèle SysML/UML**: le projet Rhapsody ouvert, qui contient les éléments manipulés par le plugin (acteurs «**stakeholder**», classes «**system**», flux «**valuearc**», diagrammes «**SVNDiagram**»). Le plugin lit et modifie ce modèle directement via l'API Rhapsody.
- **L'utilisateur**: déclenche les commandes du plugin depuis le menu de Rhapsody ou en ligne de commande, et crée manuellement les éléments SVN dans le modèle.

Le schéma ci-dessous représente ces interactions :



Le plugin n'a pas de persistance propre : toutes les données (scores, stéréotypes, tags) sont stockées directement dans le modèle Rhapsody. Il ne communique avec aucun service réseau ou base de données externe.

Architecture en couches

Le code est organisé selon une architecture en couches :

Couche	Classes	Rôle
Point d'entrée	SVNPlugin, Main (permet de lancer le programme sans Rhapsody)	Initialisation du plugin, connexion à Rhapsody
Contrôleur réactif	Listener	Écoute les événements Rhapsody (afterAddElement, onElementsChanged) et orchestre les mises à jour
Services (logique métier)	CalculationService	Calcul d'importance SVN, orchestration des stratégies
Stratégies de calcul	ValueLoopStrategy, ArcSumStrategy	Algorithmes interchangeables (pattern Strategy)
Modèles	Stakeholder, ValueArc, SVNSystem, ValueLoop	Wrappers des interfaces Rhapsody
Wrapper Rhapsody	RhapsodyWrapper, RhapsodyElementUpdater	Accès bas niveau à l'API Rhapsody et écriture des résultats dans le modèle
Constantes	SVNConstants	Centralisation des noms de stéréotypes et de tags

Le plugin adopte une architecture **réactive**: il ne repose pas sur des commandes manuelles déclenchées par l'utilisateur, mais sur un `Listener` enregistré auprès de Rhapsody qui réagit automatiquement aux modifications du modèle. Lorsqu'un arc «valuearc» est ajouté, qu'un de ses tags (`benefitRanking`, `supplyImportance`) est modifié, ou qu'un élément est supprimé, le `Listener` déclenche aussitôt le recalcul des scores d'importance et la mise à jour des labels dans le diagramme — sans intervention de l'utilisateur.

Caractéristiques des utilisateurs

Ce plugin s'adresse principalement à des **ingénieurs systèmes** (professionnels ou étudiants).

Caractéristique	Description
Domaine	Ingénierie système, modélisation SysML/UML
Outil maîtrisé	IBM Rhapsody (utilisateur régulier)
Connaissance méthode	Familier avec la notion de Stakeholder Value Network (SVN), les équations de Cameron (2007) et les matrices de score INCOSE 2018
Fréquence d'utilisation	Occasionnelle
Compétences informatiques	Intermédiaires à avancées (capable d'utiliser un outil de modélisation professionnel)



L'utilisateur n'a pas besoin de connaître Java ni l'API Rhapsody pour utiliser le plugin : toutes les interactions passent par le menu graphique de Rhapsody et par le panneau de propriétés natif de Rhapsody.

Les contraintes principales de développement

Contraintes techniques

Composant	Contrainte
Langage	Java 8
JDK	JDK 1.8
IBM Rhapsody	Version 9.0 obligatoire (le JAR <code>rhapsody.jar</code> utilisé est spécifique à cette version)
Système d'exploitation	Windows uniquement
Outil de build	Maven 3.x
Dépendance externe	<code>rhapsody.jar</code> fourni par IBM, à placer dans <code>lib/</code>

Méthode de travail

Le projet a été mené selon une **méthodologie Agile Scrum**, structurée en **3 sprints de 3 semaines** (du 14 avril au 16 juin 2026).

Cette méthode consiste à découper la période de travail en **sprints** de 2 semaines. Ce découpage en courte période permet d'explorer rapidement des fonctionnalités et de réagir rapidement aux imprévus et changements.

Rôle	Membre
Product Owner	Guillaume Retter
Scrum Master	Arnaud Michel
Développeurs	Hugo Charcot, Benjamin Groisne, Antoine Laurant

Chaque sprint est encadré par un sprint planning et une rétrospective. Le backlog du projet est géré sur **ClickUp**.

Les rétrospectives de fin de sprint ont également permis de détecter et de corriger rapidement les dettes techniques accumulées, notamment lors du Sprint 3 où une refactorisation profonde de l'architecture (introduction du **Listener** et du pattern Strategy) a été planifiée et livrée dans les délais.

Normes et standards

Le calcul d'importance des parties prenantes est conforme aux **équations SVN définies par Cameron (2007)** (["Références bibliographiques" in "Références"](#)), qui formalisent la notion de *value loop* et le calcul du score d'importance relatif de chaque stakeholder. La pondération des arcs «valuearc» repose sur la **matrice de score publiée en Figure 3 de l'INCOSE 2018** (["Références bibliographiques" in "Références"](#)), qui associe à chaque combinaison de **benefitRanking** et **supplyImportance** une valeur numérique comprise entre 0.1 et 0.95.

La modélisation des entités SVN dans Rhapsody s'appuie sur des **stéréotypes SysML** définis dans un profil dédié (**SVNProfile**), conformément aux pratiques de profilage UML/SysML recommandées par l'OMG.

Le code source suit les **conventions de codage Java standard** (nommage en camelCase, constantes en UPPER_SNAKE_CASE, séparation des responsabilités en packages).

Design patterns utilisés

Trois patterns de conception sont mis en œuvre dans l'architecture du plugin :

Patte rn	Classe principale	Rôle
Singleton	Logger	Instance unique partagée par tous les composants pour la journalisation
Strategy	ICalculationStrategy	Interchangeabilité des algorithmes de calcul (ValueLoopStrategy/ArcSumStrategy) sans modification du code appelant
Observer	Listener (RPApplicationListener)	Souscription aux événements Rhapsody (afterAddElement, onElementsChanged) pour le recalcul automatique

Hypothèses de travail

Lors du développement du plugin, les hypothèses suivantes ont été formulées concernant l'environnement d'exécution, les données d'entrée et les comportements attendus. Le respect de ces hypothèses est crucial pour le bon fonctionnement du plugin. En cas de non-respect, des impacts fonctionnels ou des erreurs peuvent survenir.

Hypothèse	Impact si non respectée
IBM Rhapsody 9.0 est ouvert avec un projet actif au moment du lancement	Le plugin ne peut pas se connecter à l'API — toutes les commandes échouent
Un package <code>Default</code> existe dans le projet Rhapsody	Les commandes de création et de nettoyage ne trouvent pas leur cible
La commande <code>SVN Configure</code> a été exécutée avant toute autre	Les stéréotypes et tags nécessaires (<code>«stakeholder»</code> , <code>«valuearc»</code> , <code>importanceScore</code> , etc.) sont absents du modèle
Des éléments <code>«stakeholder»</code> , <code>«system»</code> et <code>«valuearc»</code> ont été créés dans le modèle	<code>CalculationService</code> ne trouve aucun nœud à traiter et renvoie un résultat vide
Un arc <code>«valuearc»</code> est créé ou modifié dans un diagramme qui n'est pas un <code>«SVNDiagram»</code>	Le <code>Listener</code> ne trouve pas le diagramme SVN correspondant — <code>getSVNDiagrams()</code> retourne <code>null</code> et le recalcul est ignoré

En cas de modèle sans nœud `«system»`, le plugin bascule automatiquement sur un **calcul simplifié par somme des arcs** (`ArcSumStrategy`), ce qui constitue la principale solution de repli fonctionnelle.

2. Besoins fonctionnels

Le plugin expose **sept cas d'utilisation**, tous déclenchés par l'ingénieur système depuis le menu contextuel de Rhapsody. Ils sont regroupés en trois grandes fonctionnalités : **initialisation**, **analyse SVN**, et ****visualisation****.

 **Remarque:** Il est également possible de déclencher ses cas d'utilisation en ligne de commande lors du développement, mais l'usage principal est via le menu de Rhapsody.

Les différents cas d'utilisation sont répartis en quatre groupes :

Groupe	Cas d'utilisation
Initialisation	UC1 — SVN Configure, UC7 — SVN Clean
Édition du modèle	UC2 — SVN Create Arc, UC3 — SVN Edit Arc
Analyse SVN	UC4 — SVN Calculate
Visualisation	UC5 — SVN Colorize Elements, UC6 — SVN Set Arc Label

UC1 — Configurer le profil SVN

Champ	Contenu
Nom	SVN Configure
Description	Crée ou recrée le profil SVNProfile dans le projet Rhapsody actif. Initialise les stéréotypes («stakeholder», «system», «valuearc», «SVNDiagram»), les types énumérés (BenefitRanking, SupplyImportance) et les tags associés.
Entrées et provenances	Projet Rhapsody actif (via IRPApplication.activeProject())
Traitement	ProfileService.configureProfile(true) — supprime l'ancien profil s'il existe, puis recrée tous les éléments du profil.
Sorties	Le profil SVNProfile est présent dans le projet. Message console [SVN] Plugin initialisé. Aucun retour visuel à l'utilisateur.
Classe ou instance associée	SVNPlugin (init), RhapsodyWrapper (création des stéréotypes et tags)

UC2 — Créer un arc de valeur

Champ	Contenu
Nom	SVN Create Arc
Description	Crée un arc «valuearc» entre deux éléments sélectionnés dans le diagramme Rhapsody.
Entrées et provenances	Deux éléments sélectionnés graphiquement dans Rhapsody (source et cible de l'arc)
Traitement	DiagramService.createArcBetweenSelected() — récupère la sélection courante via IRPApplication.getSelectedGraphElements(), vérifie que les deux nœuds portent un stéréotype SVN («stakeholder» ou «system»), crée un IRPFlow avec le stéréotype «valuearc», puis l'insère graphiquement dans le diagramme.
Sorties	Un arc «valuearc» apparaît dans le diagramme entre les deux éléments sélectionnés. Message console confirmant la création.
Classe ou instance associée	Listener.afterAddElement(), ValueArc (initialisation des tags par défaut), RhapsodyElementUpdater.updateArcLabel()

UC3 — Éditer les pondérations d'un arc

Champ	Contenu
Nom	SVN Edit Arc
Description	Permet à l'utilisateur de renseigner les deux tags de pondération d'un arc «valuearc» sélectionné : benefitRanking et supplyImportance.
Entrées et provenances	A définir
Traitement	Le Listener.onElementsChanged() détecte la modification d'un tag benefitRanking ou supplyImportance via le panneau de propriétés de Rhapsody. Il écrit la valeur mise à jour dans le modèle et déclenche le recalcul.
Sorties	Les tags benefitRanking et supplyImportance de l'arc sont mis à jour. Message d'avertissement si l'élément sélectionné n'est pas un «valuearc».
Classe ou instance associée	A définir

Valeurs possibles :

Tag	Valeurs
benefitRanking	MIGHT_BE, SHOULD_BE, MUST_BE
supplyImportance	LOW, MEDIUM, HIGH

UC4 — Calculer l'importance des stakeholders

Champ	Contenu
Nom	SVN Calculate
Description	Calcule le score d'importance normalisé de chaque «stakeholder» du modèle en appliquant les équations SVN (Cameron 2007 ("Références bibliographiques" in "Références")/INCOSE 2018 ("Références bibliographiques" in "Références")).
Entrées et provenances	Projet Rhapsody actif contenant des éléments «stakeholder», «system» et «valuearc» avec leurs tags benefitRanking et supplyImportance renseignés
Traitement	CalculationService.calculateImportance() — parcourt récursivement le modèle, construit le graphe des arcs, recherche les <i>value loops</i> par DFS depuis le nœud «system», puis calcule l'importance relative de chaque stakeholder (Équation 2, Cameron 2007 ("Références bibliographiques" in "Références")). En l'absence de nœud «system», bascule sur un calcul simplifié par somme des arcs.
Sorties	Le tag importanceScore (valeur entre 0 et 1) de chaque «stakeholder» est mis à jour dans le modèle. Messages console détaillant les loops trouvés et les scores calculés.
Classe ou instance associée	CalculationService, ValueLoopStrategy/ArcSumStrategy, ValueLoop, SearchState, RhapsodyElementUpdater.updateStakeholderImportance()

Algorithme de calcul (méthode principale) :

1. Construire le graphe orienté des arcs «valuearc»
2. Trouver tous les cycles (value loops) depuis le nœud «system» par DFS
3. **Équation 1:** score d'un loop = produit des scores de ses arcs
4. **Équation 2:** importance(stakeholder S) = Σ scores des loops contenant S / Σ scores de tous les loops

Matrice de score des arcs (INCOSE 2018 (["Références bibliographiques" in "Références"](#)), Figure 3) :

supplyImportance ↓ \ benefitRanking →	MIGHT_BE	SHOULD_BE	MUST_BE
HIGH	0.30	0.50	0.95
MEDIUM	0.20	0.40	0.80
LOW	0.10	0.20	0.40

UC5 — Coloriser les stakeholders

⚠ Remarque: Ce cas d'utilisation est en cours de développement. Il est déjà intégré dans notre conception, mais n'est pas encore disponible dans le plugin.

Champ	Contenu
Nom	SVN Colorize Elements
Description	Applique une couleur de remplissage à chaque nœud «stakeholder» et à chaque arc «valuearc» dans les diagrammes SVN, en fonction de leur rang d'importance calculé.
Entrées et provenances	Modèle Rhapsody avec des «stakeholder» dont le tag importanceScore est renseigné (nécessite l'exécution préalable de UC4)
Traitement	Trie les stakeholders par score décroissant, les divise en trois tiers, applique les couleurs rouge/orange/jaune selon l'importance via RhapsodyWrapper (propriété graphique FillColor). Applique la même logique de tiers aux arcs «valuearc» (propriété LineColor).
Sorties	Les nœuds «stakeholder» et les arcs «valuearc» sont colorisés dans les diagrammes SVN (rouge = plus importants, orange = importants, jaune = moins importants).
Classe ou instance associée	RhapsodyElementUpdater.updateStakeholderImportance(), Stakeholder, ValueArc, RhapsodyWrapper

Code couleur appliqué :

Tier	Couleur	Signification
Tiers supérieur (1/3 le plus important)	Rouge (#FF4444)	Stakeholders/Arcs critiques
Tiers médian	Orange (#FFA500)	Stakeholders/Arcs importants
Tiers inférieur	Jaune (#FFFF00)	Stakeholders/Arcs secondaires

UC6 — SVN Set Arc Label

Champ	Contenu
Nom	SVN Set Arc Label
Description	Met à jour manuellement le label d'un arc «valuearc» sélectionné pour afficher son score numérique calculé depuis ses tags benefitRanking et supplyImportance.
Entrées et provenances	Arc «valuearc» sélectionné dans Rhapsody avec ses tags benefitRanking et supplyImportance renseignés
Traitement	RhapsodyElementUpdater.updateArcLabel() — calcule le score numérique de l'arc via ValueArc.getScore() et met à jour son displayName. En cas d'échec, tente une mise à jour via IRPGraphElement.setGraphicalPropertyOfText().
Sorties	L'arc affiche son score numérique dans le diagramme (ex. : 0.95). Message console confirmant la mise à jour.
Classe ou instance associée	RhapsodyElementUpdater.updateArcLabel(), ValueArc.getScore()

UC7 — Nettoyer le modèle

Champ	Contenu
Nom	SVN Clean
Description	Supprime tous les éléments SVN du package Default du projet Rhapsody (stakeholders, systems, valuearcs, diagrammes SVN) ainsi que le profil SVNProfile. Remet le modèle dans un état vierge.
Entrées et provenances	Projet Rhapsody actif
Traitement	ProfileService.cleanDefaultPackage() puis ProfileService.deleteProfile() — collecte tous les éléments SVN avant suppression, supprime d'abord les arcs, puis les nœuds, puis les diagrammes, dans le bon ordre pour éviter les erreurs de dépendance.
Sorties	Le modèle ne contient plus d'éléments SVN. Le profil SVNProfile est supprimé. Message console indiquant le nombre d'éléments supprimés.
Classe ou instance associée	RhapsodyWrapper, SVNConstants

3. Spécification des structures de données

⚠ Le plugin RhapsodySVN ne possède pas de base de données propre. Toutes les données sont stockées directement dans le **modèle Rhapsody** sous forme de stéréotypes, de tags et de types énumérés. Cette section décrit les entités manipulées, leurs relations et leurs attributs.

Modèle d'entités

```
SVNProfile
├─ Stéréotype «stakeholder» (métaclasse Actor)
│   └─ tag: importanceScore (Float)
├─ Stéréotype «system» (métaclasse Class)
├─ Stéréotype «valuearc» (métaclasse Flow)
│   └─ tag: benefitRanking (BenefitRanking)
│       └─ tag: supplyImportance (SupplyImportance)
├─ Stéréotype «SVNDiagram» (métaclasse ObjectModelDiagram)
├─ Type BenefitRanking (Enumeration)
│   └─ MIGHT_BE
│   └─ SHOULD_BE
│   └─ MUST_BE
└─ Type SupplyImportance (Enumeration)
    └─ LOW
    └─ MEDIUM
    └─ HIGH
```

Entité : «stakeholder»

Propriété	Valeur
Nom	stakeholder
But	Représente une partie prenante du système (acteur humain ou organisation) dont on souhaite évaluer l'importance relative dans le réseau de valeur
Métaclass e Rhapsody	Actor (IRPActor)
Profil	SVNProfile
Tag	importanceScore (Float, entre 0 et 1) — score d'importance calculé par UC4 ("UC4 — Calculer l'importance des stakeholders" in "2. Besoins fonctionnels")
Relations	Peut être relié à d'autres «stakeholder» ou à un «system» via des arcs «valuearc»

Entité : «system»

Propriété	Valeur
Nom	system
But	Représente le système central du réseau SVN. C'est le nœud de départ et d'arrivée des <i>value loops</i> utilisées dans le calcul de Cameron (2007). Un seul nœud «system» est attendu par modèle.
Métaclasses Rhapsody	Class (IRPCClass)
Profil	SVNProfile
Tags	Aucun tag spécifique
Relations	Connecté aux «stakeholder» via des «valuearc»

Entité : «valuearc»

Propriété	Valeur
Nom	valuearc
But	Représente un flux de valeur orienté entre deux entités du réseau SVN (stakeholder ↔ system ou stakeholder ↔ stakeholder). Chaque arc est pondéré par deux critères.
Métaclasses Rhapsody	Flow (IRPFlow)
Profil	SVNProfile
Tag benefitRanking	Type BenefitRanking — importance du bénéfice apporté par cet arc du point de vue du destinataire (MIGHT_BE, SHOULD_BE, MUST_BE)
Tag supplyImportance	Type SupplyImportance — capacité du fournisseur à satisfaire ce flux de valeur (LOW, MEDIUM, HIGH)
Relations	Orienté entre deux entités SVN : end1 (source) et end2 (cible)

Entité : «SVNDiagram»

Propriété	Valeur
Nom	SVNDiagram
But	Diagramme Rhapsody dédié à la représentation visuelle d'un réseau SVN. Il regroupe les nœuds «stakeholder» et «system» et les arcs «value arc».
Métaclass e Rhapsod y	ObjectModelDiagram (IRPObjectModelDiagram)
Profil	SVNProfile
Propriété	DrawingToolbar configurée pour exposer les outils stakeholder, system et valuearc
Relations	Contient les représentations graphiques (IRPGraphElement) des entités SVN

Type énuméré : BenefitRanking

Propriété	Valeur
Nom	BenefitRanking
But	Qualifie l'importance du bénéfice que représente un arc «valuearc» pour son destinataire
Littérature	MIGHT_BE, SHOULD_BE, MUST_BE (ordre croissant d'importance)
Utilisation	Tag benefitRanking du stéréotype «valuearc»

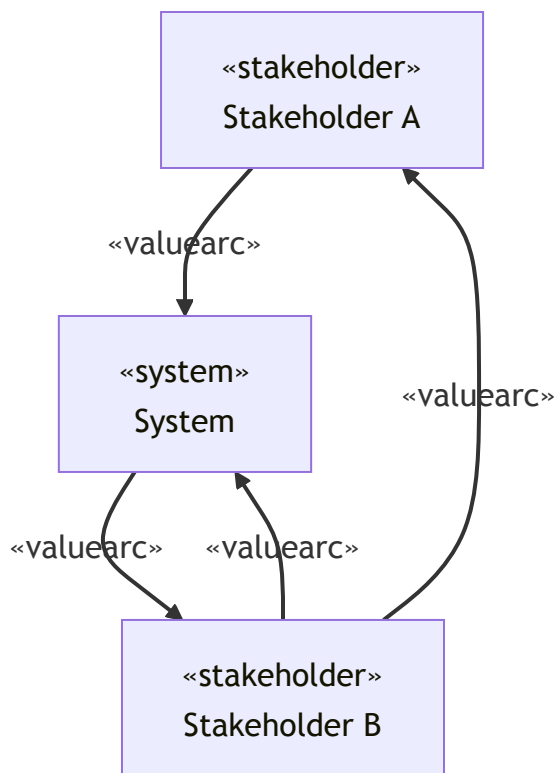
Type énuméré : SupplyImportance

Propriété	Valeur
Nom	SupplyImportance
But	Qualifie la capacité du fournisseur à satisfaire le flux de valeur représenté par un arc
Littéraux	LOW, MEDIUM, HIGH (ordre croissant de capacité)
Utilisation	Tag supplyImportance du stéréotype «valuearc»

Profil SVNProfile

Propriété	Valeur
Nom	SVNProfile
But	Profil SysML/UML créé dans le projet Rhapsody actif, qui regroupe tous les stéréotypes et types nécessaires au fonctionnement du plugin
Contenu	Stéréotypes «stakeholder», «system», «valuearc», «SVNDiagram»; types BenefitRanking, SupplyImportance
Gestion	Créé ou recréé par UC1. Supprimé par UC8. Stocké dans le modèle Rhapsody (pas de fichier externe).

Relations entre entités



Un réseau SVN typique comprend :

- Un nœud «system» central
- Plusieurs nœuds «stakeholder» autour
- Des arcs «valuearc» orientés formant des cycles (value loops) passant par le système

4. Spécifications des interfaces externes

Interface matériel / logiciel

Composant	Configuration minimale
Système d'exploitation	Windows (7, 10 ou 11) — Windows uniquement
Ressources matérielles	Ne consomme pas plus que Rhapsody sans plugin
JDK	JDK 1.8 (Java 8) installé et accessible dans le PATH

Interface logiciel / logiciel

IBM Rhapsody 9.0

Propriété	Valeur
Nom	IBM Rational Rhapsody
Version	9.0 (obligatoire — l'API COM utilisée est spécifique à cette version)
Provenance	Fourni par IBM. <i>Nécessite une licence spécifique</i>
But d'utilisation	Application hôte du plugin. Fournit l'API Java via <code>rhapsody.jar</code> .
Interface	<code>IRPApplication</code> , <code>IRPPProject</code> , <code>IRPPProfile</code> , <code>IRPFlow</code> , <code>IRPActor</code> , <code>IRPCollection</code> , <code>RPUUserPlugin</code> , <code>RhapsodyAppServer</code>

Le JAR `rhapsody.jar` est inclus comme dépendance locale dans `lib/rhapsody.jar` et déclaré dans `pom.xml`.

Maven 3.x

Propriété	Valeur
Nom	Apache Maven
Version	3.x
But	Outil de build et de gestion des dépendances
Usage	Compilation du projet, packaging en JAR (<code>mvn package</code>)

Interface Homme / logiciel

Interaction via Rhapsody

Le plugin ne possède **aucune interface graphique propre**. Il est entièrement réactif : l'utilisateur interagit exclusivement via les outils natifs de Rhapsody, et le plugin répond automatiquement à ces actions.

Action utilisateur dans Rhapsody	Réaction automatique du plugin
Création d'un élément «valuearc» dans le diagramme	Initialisation des tags par défaut, mise à jour du label, recalcul des scores
Modification du tag <code>benefitRanking</code> ou <code>supplyImportance</code> via le panneau de propriétés Rhapsody	Mise à jour du label de l'arc, recalcul des scores d'importance
Suppression d'un élément SVN	Recalcul des scores d'importance

Messages console

Le plugin utilise un `Logger` singleton qui écrit sur la sortie standard. Trois niveaux sont définis :

Niveau	Format	Condition d'affichage
Info	<code>[SVN] <message></code>	Uniquement si le mode debug est activé
Avertissement	<code>[SVN][WARN] <message></code> (jaune)	Uniquement si le mode debug est activé
Erreur	<code>[SVN][ERROR] <message></code> (rouge)	Toujours affiché

Le mode debug s'active en passant l'argument `debug` au lancement via `Main`, ou via `logger.setDebug(true)`.

Exemples de messages produits :

- `[SVN] Plugin init success` — à l'ouverture du plugin
- `[SVN] Stakeholder found : Stakeholder1` — lors du calcul UC4
- `[SVN] Value loops found : 3` — lors du calcul UC4
- `[SVN] Importance 0.3750` — score calculé pour un stakeholder
- `[SVN][ERROR] Error in afterAddElement: <détail>` — en cas d'exception dans le Listener

Spécification des messages d'erreur

Situation	Message affiché
Projet Rhapsody non actif au démarrage	[SVN] Project is null
Aucun stakeholder trouvé dans le diagramme	[SVN] No stakeholders found.
Aucun arc «valuearc» trouvé	[SVN] No arcs, can't calculate.
Aucun value loop détecté (fallback)	[SVN] No «SVNsystem» found — switching to arc sum strategy.
Exception dans afterAddElement	[SVN][ERROR] Error in afterAddElement: <détail>
Exception dans onElementsChanged	[SVN][ERROR] Error on onElementsChanged : <détail>

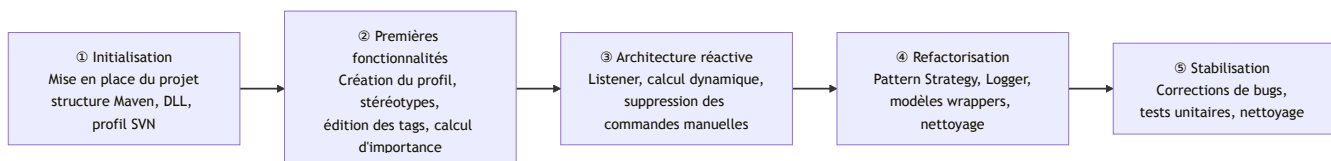
5. Avancement du projet

⚠ Note: partie à modifier

Ce chapitre constitue un mini-audit du plugin RhapsodySVN. Il évalue le niveau de maturité du projet sur cinq axes : qualité du code, maintenabilité, robustesse, sécurité et limitations connues.

Historique du développement

L'historique git révèle cinq phases de développement successives :



Phase	Commits clés
① Initialisation	core: creating java project, a02aeefe relative dll path config, feat: implement SVN profile creation
② Premières fonctionnalités	can edit Tags from ARC and stakeholder, feat: implement SVN profile configuration and importance calculation, feat: add properties to stereotypes
③ Architecture réactive	feat: add tags values views on elements, and dynamic calculation, refactor: add elements deletion handling on onElementsChanged trigger, chore: delete main commands
④ Refactorisation	feat: start extracting calculation strategies to Strategy Classes, refactor: move static methods from SVNConstant to RhapsodyWrapper, chore: move UpdateElement class to rhapsody package
⑤ Stabilisation	fix: diagram was null when changing arc values, fix: add calculation over with IRPPort, test: add unit test with Junit, chore: French comments and logs messages cleaned

Qualité du code

Le code est actuellement **en cours de refactorisation**. La version initiale du projet reposait sur des commandes manuelles déclenchées depuis le menu Rhapsody (chore: delete main commands) ; l'architecture a depuis évolué vers un modèle réactif piloté par un Listener, ce qui constitue une amélioration significative de la conception.

Les responsabilités sont bien séparées entre les différentes classes : le Listener gère la réception des événements Rhapsody, le CalculationService orchestre le calcul, les stratégies (ValueLoopStrategy, ArcSumStrategy) encapsulent les algorithmes, et RhapsodyElementUpdater centralise l'écriture des résultats dans le modèle. Cette répartition applique le **principe de responsabilité unique**.

Le **pattern Strategy** est correctement utilisé pour rendre les algorithmes de calcul interchangeables sans modifier le code appelant. Les classes modèles (Stakeholder,

ValueArc, SVNSystem) encapsulent proprement les interfaces Rhapsody et exposent une API cohérente.

L'architecture est désormais stabilisée : les sections UC du rapport ont été mises à jour pour refléter les classes actuelles (RhapsodyElementUpdater à la place de l'ancien UpdateElementService).

Maintenabilité

La maintenabilité du projet est **bonne** pour un projet académique. Plusieurs choix techniques facilitent les évolutions futures :

- **Centralisation des constantes:** tous les noms de stéréotypes, tags et littéraux sont regroupés dans SVNConstants, évitant les chaînes de caractères dispersées dans le code.
- **Isolation de l'API Rhapsody:** RhapsodyWrapper regroupe les accès bas niveau à l'API. En cas de changement de version de Rhapsody, les adaptations sont localisées dans cette seule classe.
- **Interfaces et stratégies interchangeables:** ICalculationService et ICalculationStrategy permettent d'ajouter ou de remplacer un algorithme sans toucher au Listener ni au reste de l'architecture.
- **Classes courtes et ciblées:** les classes et méthodes sont relativement concises, ce qui facilite la compréhension et la modification du code.

La documentation du code est **prévue mais pas encore complète**: les commentaires Javadoc sont absents sur la majorité des méthodes. Cette lacune sera comblée en fin de projet pour assurer une meilleure transmission du code.

Robustesse et fiabilité

Le projet dispose désormais d'une **suite de tests JUnit** couvrant les parties algorithmiques critiques :

Classe de test	Ce qui est couvert
SearchStateTest	Logique de l'état de recherche DFS (value loops)
ValueArcScoreTest	Matrice de scores INCOSE (BenefitRanking × SupplyImportance)
ValueLoopTest	Calcul du score d'un value loop (produit des arcs)
ImportanceCalculationTest	Calcul d'importance end-to-end (équations Cameron)

Ces tests couvrent la couche algorithmique pure (modèles et stratégies) sans dépendance à l'API Rhapsody, ce qui les rend exécutables en dehors de l'environnement IBM. La couche d'intégration (interaction avec Rhapsody via `IRPApplication`) reste non testée automatiquement : elle est validée uniquement par tests manuels.

Les principaux mécanismes de protection en place sont :

- Les opérations API Rhapsody sont systématiquement encadrées par des blocs `try/catch` avec log d'erreur, évitant toute exception non gérée susceptible de planter Rhapsody.
- L'absence de nœud «system» est gérée par un fallback automatique vers `ArcSumStrategy`.
- Les tags absents ou invalides sont traités avec des valeurs par défaut via `RhapsodyWrapper.initTagIfAbsent()`.
- Les notifications Rhapsody sont désactivées (`setNotifyPluginOnElementsChanged(0)`) pendant les opérations d'écriture pour éviter les boucles d'événements.

Sécurité

Le plugin présente une **surface d'attaque très limitée**. Il ne communique avec aucun service réseau, ne lit ni n'écrit aucun fichier externe, et ne manipule aucune donnée

sensible (mots de passe, données personnelles, jetons d'authentification).

Toutes les données (scores, tags, stéréotypes) sont stockées exclusivement dans le modèle Rhapsody ouvert. L'accès au plugin est restreint à la machine locale : il ne peut être déclenché que depuis IBM Rhapsody ou en ligne de commande, sans exposition d'API externe.

Les valeurs de tags reçues depuis le modèle sont traitées défensivement : toute valeur absente, nulle ou non reconnue est remplacée par une valeur par défaut sans lever d'exception. Le plugin ne fait pas confiance aux données du modèle et les valide avant usage.

Limitations connues

Limitation	Impact
Couplage à Rhapsody 9.0	Le JAR <code>rhapsody.jar</code> est spécifique à cette version. Une migration vers une version ultérieure de Rhapsody nécessiterait de revalider l'ensemble des appels API.
Windows uniquement	La DLL native <code>rhapsody.dll</code> requise par l'API Java COM n'existe que pour Windows. Le plugin ne peut pas être exécuté sur Linux ou macOS.
Couverture de tests partielle	Les tests JUnit couvrent la logique algorithmique, mais la couche d'intégration Rhapsody (<code>Listener</code> , <code>RhapsodyWrapper</code> , <code>RhapsodyElementUpdater</code>) reste non couverte — l'API Rhapsody ne peut pas être mockée sans une vraie instance IBM.
Usage de IRPDependency au lieu de constructs SysML	Les arcs «valuearc» sont modélisés comme des <code>IRPDependency</code> (dépendance UML générique) plutôt que comme des <code>IRPConnector</code> ou des associations de blocs SysML. Cela s'éloigne de la sémantique IBD/BDD attendue pour un diagramme SVN correct en SysML.

Références

Références bibliographiques

Référence	Description
Cameron (2007)	B. Cameron, <i>"Stakeholder Value Network Analysis"</i> , INCOSE 2007. Définit la méthode SVN et les équations de calcul d'importance relative des parties prenantes (Équations 1 et 2).
INCOSE 2018	International Council on Systems Engineering, <i>Systems Engineering Handbook</i> , version 4, 2018. La Figure 3 définit la matrice de score des arcs utilisée pour calculer le poids de chaque «valuearc».
IEEE Std 830-1998	IEEE, <i>Recommended Practice for Software Requirements Specifications</i> , 1998. Norme de référence pour la structure du présent document de spécification.
Sommerville (1989)	I. Sommerville, <i>"Software Engineering"</i> , 3rd edn. Wokingham : Addison-Wesley, 1989.
Satzinger et al. (2002)	J. Satzinger, R.B. Jackson, S. Burd, <i>"Analyse et conception des systèmes d'information"</i> , Editions Rynold Goulet, 2002.
Jacobson et al. (1999)	I. Jacobson, G. Booch, J. Rumbaugh, <i>"The Unified Software Development Process"</i> , Addison-Wesley, 1999.
Koukam (2006)	A. Koukam, <i>"Software Engineering"</i> , Course "Introduction to Software Engineering", University of Shanghai, China, 2006.

Documentation technique

Référence	Description
IBM Rhapsody API	Documentation de l'API Java COM d'IBM Rational Rhapsody 9.0. Interfaces <code>IRPApplication</code> , <code>IRPPProject</code> , <code>IRPPProfile</code> , <code>IRPFlow</code> , <code>IRPActor</code> , <code>RPUserPlugin</code> , etc.
Java SE 8 API	Documentation officielle Oracle du JDK 1.8, incluant <code>javax.swing</code> (<code>JOptionPane</code> , <code>JColorChooser</code>).
Apache Maven	Documentation officielle Maven 3.x. Structure de projet <code>src/main/java</code> , gestion des dépendances <code>pom.xml</code> .

Index

Index alphabétique des termes

Terme	Section(s)
API	§1, §2, §5
Arc de valeur («valuearc»)	§2, §3 (UC2, UC3, UC6, UC7), §4, §5
BenefitRanking	§3 (UC3, UC4), §4, Annexe B
CalculationService	§2, §3 (UC4)
Cameron (2007)	§3 (UC4), §7, §8, Annexe A
DiagramService	§2, §3 (UC2, UC5, UC6, UC7)
DFS (Depth-First Search)	§3 (UC4), §6, Annexe A
Diagramme SVN («SVNDiagram»)	§2, §3 (UC2, UC5, UC6, UC7), §4
importanceScore	§3 (UC4, UC5), §4
IBM Rhapsody	§1, §2, §5, §7
INCOSE 2018	§3 (UC4), §7, §8, Annexe B
IRPActor	§4, §5
IRPApplication	§2, §5
IRPCollection	§1, §4
IRPFlow	§3 (UC2, UC3), §4, §5

IRPProject	§2, §5
Java 8 / JDK 1.8	§2, §5, §7
JColorChooser	§3 (UC6), §5
JOptionPane	§3 (UC3), §5
Maven	§2, §5, §7
Nœud système («system»)	§2, §3 (UC4), §4
ProfileService	§2, §3 (UC1, UC8)
RhapsodyWrapper	§2, §7
RPUUserPlugin	§1, §2
supplyImportance	§3 (UC3, UC4), §4, Annexe B
SupplyImportance	§3 (UC3, UC4), §4, Annexe B
SVNCalculateCommand	§2, §3 (UC4)
SVNCleanCommand	§2, §3 (UC8)
SVNConfigureCommand	§2, §3 (UC1)
SVNConstants	§2, §7
SVNPlugin	§1, §2, §5

SVNProfile	§2, §3 (UC1, UC8), §4
Stakeholder («stakeholder»)	§2, §3 (UC4, UC5), §4
Stéréotype	§1, §2, §3, §4
SysML	§1, §2
Tag	§1, §3, §4
Value loop	§1, §3 (UC4), §6, Annexe A
Windows	§2, §5

Index des fonctions et classes

Classe / Méthode	Rôle	Section(s)
SVNPlugin	Point d'entrée du plugin, route les commandes	§2, §5
SVNConfigureCommand.run()	Lance la configuration du profil SVN	§3 (UC1)
SVNCreateArcCommand.run()	Crée un arc «valuearc» entre deux éléments sélectionnés	§3 (UC2)
SVNEditArcCommand.run()	Édite les pondérations d'un arc sélectionné	§3 (UC3)
SVNCalculateCommand.run()	Lance le calcul d'importance SVN	§3 (UC4)
SVNColorizeStakeholdersCommand.run()	Colorise les stakeholders par rang	§3 (UC5)
SVNArcColorCommand.run()	Applique une couleur à un arc	§3 (UC6)
SVNLabelArcCommand.run()	Met à jour les labels d'arcs	§3 (UC7)
SVNCleanCommand.run()	Nettoie le modèle SVN	§3 (UC8)
ProfileService.configureProfile()	Crée/recrée le profil SVNProfile	§3 (UC1)
ProfileService.cleanDefaultPackage()	Supprime les éléments SVN du package Default	§3 (UC8)
CalculationService.calculateImportance()	Orchestre le calcul d'importance	§3 (UC4)

<code>CalculationService.calculateByValueLoops()</code>	Calcul principal par DFS	§3 (UC4), Annexe A
<code>CalculationService.calculateByArcSum()</code>	Calcul simplifié (fallback)	§3 (UC4)
<code>DiagramService.createArcBetweenSelected()</code>	Crée un arc entre éléments graphiques	§3 (UC2)
<code>DiagramService.colorizeStakeholdersByRank()</code>	Colorise les nœuds par tier	§3 (UC5)
<code>DiagramService.setArcColor()</code>	Change la couleur d'un arc	§3 (UC6)
<code>DiagramService.updateArcLabels()</code>	Met à jour les labels d'arcs	§3 (UC7)
<code>RhapsodyWrapper.hasStereotype()</code>	Vérifie si un élément porte un stéréotype donné	§2, §3
<code>RhapsodyWrapper.getOrCreateStereotype()</code>	Crée ou récupère un stéréotype	§3 (UC1)
<code>RhapsodyWrapper.getOrCreateEnumType()</code>	Crée ou récupère un type énuméré	§3 (UC1)

Annexes

Annexe A — Méthode SVN et équations de Cameron (2007)

Présentation de la méthode

La méthode **Stakeholder Value Network (SVN)** est une approche d'ingénierie système permettant d'analyser les relations de valeur entre les parties prenantes d'un système. Elle représente ces relations sous forme d'un réseau orienté (graphe) où :

- Les **nœuds** sont des parties prenantes («stakeholder») et le système central («system»)
- Les **arcs** («valuearc») représentent des flux de valeur orientés entre deux entités

Scores des arcs

Chaque arc est caractérisé par deux attributs :

- **benefitRanking**: importance du bénéfice apporté par le flux du point de vue du destinataire (MIGHT_BE, SHOULD_BE, MUST_BE)
- **supplyImportance**: capacité du fournisseur à assurer ce flux (LOW, MEDIUM, HIGH)

Le score d'un arc est obtenu par la matrice INCOSE 2018 (voir Annexe B).

Notion de Value Loop

Un **value loop** est un cycle dans le graphe SVN qui commence et se termine sur le nœud «system». Il représente un circuit de création de valeur impliquant le système et un sous-ensemble de parties prenantes.

Exemple : system → stakeholder_A → stakeholder_B → system

Équation 1 — Score d'un value loop

Le score d'un value loop L est le **produit des scores de ses arcs**:

$$\text{score}(L) = \prod \text{score}(\text{arc}_i) \quad \text{pour tout } \text{arc}_i \text{ appartenant à } L$$

Cette formule traduit le fait qu'un loop n'est efficace que si tous ses arcs sont de bonne qualité.

Équation 2 — Importance d'un stakeholder

L'importance relative d'un stakeholder S est calculée comme le **rapport entre la somme des scores des loops contenant S et la somme des scores de tous les loops**:

$$\text{importance}(S) = \frac{\sum \text{score}(L_i) \text{ [pour } L_i \text{ contenant } S]}{\sum \text{score}(L_j) \text{ [tous } L_j]}$$

Le score obtenu est normalisé entre 0 et 1. Un stakeholder impliqué dans de nombreux loops de haute valeur obtient un score proche de 1.

Fallback — Calcul simplifié par somme des arcs

En l'absence de nœud «system», aucun value loop ne peut être détecté. Le plugin bascule sur un calcul simplifié :

$$\begin{aligned} \text{score_brut}(S) &= \sum \text{score}(\text{arc}_i) \text{ [pour tout } \text{arc}_i \text{ adjacent à } S] \\ \text{importance}(S) &= \text{score_brut}(S) / \sum \text{score_brut}(S_j) \text{ [tous les stakeholders]} \end{aligned}$$

Ce calcul est moins précis que la méthode principale mais reste fonctionnel pour des modèles sans système central défini.

Annexe B — Matrice de score des arcs (INCOSE 2018, Figure 3)

La matrice ci-dessous définit le score numérique associé à chaque combinaison (benefitRanking, supplyImportance). Elle est implémentée dans CalculationService.getArcScore().

supplyImportance ↓ \ benefitRanking →	MIGHT_BE	SHOULD_BE	MUST_BE
HIGH	0.30	0.50	0.95
MEDIUM	0.20	0.40	0.80
LOW	0.10	0.20	0.40

Lecture de la matrice :

- Un arc dont le bénéfice est critique (MUST_BE) et dont la capacité de fourniture est élevée (HIGH) obtient le score maximal de **0.95**
- Un arc dont le bénéfice est optionnel (MIGHT_BE) avec une faible capacité de fourniture (LOW) obtient le score minimal de **0.10**
- La valeur par défaut (tags absents ou invalides) est **0.20** (équivalent à MIGHT_BE / MEDIUM)

Annexe C — Structure du projet Maven

```

RhapsodySVN/
├── pom.xml                    # Configuration Maven (Java
8, dépendance rhapsody.jar)
├── lib/
│   ├── rhapsody.jar          # API IBM Rhapsody 9.0
│   (dépendance locale)
│   └── 64Bit/
│       └── rhapsody.dll       # Bibliothèque native Windows
└── src/main/java/fr/utbm/RhapsodySVN/
    ├── SVNPlugin.java        # Point d'entrée plugin
    (RPUUserPlugin)
    ├── SVNConfigureCommand.java # UC1 – Configurer le profil
    ├── SVNCreateArcCommand.java # UC2 – Créer un arc
    └── SVNEditArcCommand.java   # UC3 – Éditer les
pondérations

```

```

    └─ SVNCalculateCommand.java      # UC4 – Calculer l'importance
    └─ SVNColorizeStakeholdersCommand.java # UC5 – Coloriser les
stakeholders
    └─ SVNArcColorCommand.java      # UC6 – Coloriser un arc
    └─ SVNLabelArcCommand.java      # UC7 – Mettre à jour les
labels
    └─ SVNCleanCommand.java         # UC8 – Nettoyer le modèle
    └─ Main.java                    # Point d'entrée ligne de
commande
    └─ constants/
        └─ SVNConstants.java        # Centralisation des
constantes
    └─ rhapsody/
        └─ RhapsodyWrapper.java     # Wrapper bas niveau API
Rhapsody
    └─ service/
        └─ ProfileService.java      # Gestion du profil
SVNProfile
    └─ CalculationService.java      # Algorithme de calcul SVN
    └─ DiagramService.java          # Opérations graphiques sur
les diagrammes

```

Annexe D — Procédure d'installation et de premier lancement

1. **Pré-requis:** IBM Rhapsody 9.0 installé sur Windows, JDK 1.8, Maven 3.x
2. **Compilation:** `mvn clean package` — produit `target/RhapsodySVN-1.0-SNAPSHOT.jar`
3. **Déploiement:** placer le JAR dans le répertoire plugins de Rhapsody et configurer le fichier de configuration Rhapsody pour déclarer `fr.utbm.RhapsodySVN.SVNPlugin` comme plugin actif
4. **Premier lancement:**
 - Ouvrir IBM Rhapsody avec un projet actif
 - Exécuter `SVN Configure` depuis le menu pour initialiser le profil

- Créer les éléments «stakeholder», «system» et «valuearc» manuellement dans le diagramme
- Renseigner les pondérations via SVN Edit Arc (UC3) pour chaque arc
- Lancer SVN Calculate (UC4) pour calculer les scores
- Visualiser les résultats via SVN Colorize Stakeholders (UC5) et SVN Update Arc Labels (UC7)